Group Weekly Progress Report
Week 2
Complete Simulation

Project Title:

Autonomous 3D Object Scanning and Point Cloud Visualization using Kobuki Qbot

Date: 07/04/2026

Team Name : SLAM

- **K.G.D.S.Ranasinghe** 230522H
- **R.I.B.S.P.Rathnamalala** 230541N
- **H.M.S.B.B.Herath** 230244G

1. Planned Objectives vs. Actual Progress

According to our original project proposal, Week 2 was designated for "Math & Core Drivers". The planned objectives included developing the mathematical model for the circular radius, converting raw depth data into PointCloud2 messages, and establishing basic message passing between the physical Qbot and a remote PC.

However, because we made the strategic decision to ditch our old Lab 06 code and build a custom architecture from scratch, we shifted our immediate focus. Instead of partial hardware testing, we pushed forward and **fully completed the software and simulation phase of the project this week**. We successfully implemented the mathematical logic, the control kinematics, and the RViz visualization pipeline within Ignition Gazebo.

Because we have already implemented and verified all the core logic in simulation, moving forward will be significantly easier. Our next major hurdle is simply bypassing Gazebo and feeding real-world sensor data into our existing RViz setup.

2. Revised Project Timeline & Future Goals

Having completed the simulation phase ahead of schedule, we have restructured our weekly responsibilities for the remainder of the project:

- Next 4 Weeks (Weeks 3 - 6): Hardware Implementation & Testing. We will dedicate this entire block to programming the Raspberry Pi 4, interfacing with the physical Kobuki Qbot actuators, configuring the physical 2D LiDAR and RGB-D camera, and adjusting our proven simulation scripts to operate on the physical hardware.
- Final 2 Weeks (Weeks 7 - 8): Output Platform & Polish. The final weeks will be dedicated to refining the real-time visualization platform in RViz, filtering the PointCloud2 data for clarity, and finalizing the project report and demonstration materials.

3. Successful Simulation Environment Configuration

A primary objective for Week 2 was establishing a robust simulation to test our control logic before moving to the physical Raspberry Pi 4.

- Custom Ignition Gazebo World: We developed a completely new simulated environment (scan_world.world) featuring a target cylinder optimized for the RGB-D camera.
- Sensor Integration: We fully integrated the newly acquired 2D LiDAR into the robot's URDF, dedicating it to geometric SLAM and distance tracking.
- Simultaneous Mapping: By integrating the slam_toolbox into our master launch file, the robot successfully generates a live 2D occupancy grid of the simulation environment while navigating.

4. Autonomous Control Pipeline (Start, Orbit, Stop)

The core intelligence of the robot is handled by scan_logic.py, which executes a state machine consisting of three distinct phases: Target Acquisition (Start), Precision Orbiting, and Automated Termination (Stop).

Phase 1: Target Acquisition and Start Logic

To prevent the robot from wandering or locking onto the walls of the simulation environment, the scanning logic applies a strict spatial filter to the LiDAR data. Any scan returns beyond a 2.0-meter radius are explicitly ignored.

-
- Seeking Phase: The robot identifies the nearest valid point within its 2.0-meter scanning radius. It calculates the angular error between its current heading and the target, and applies proportional control to steer toward the object at a linear velocity of 0.15 m/s.
- Orbit Threshold: The seeking phase continues until the robot reaches a predefined distance threshold (R + 0.15m).

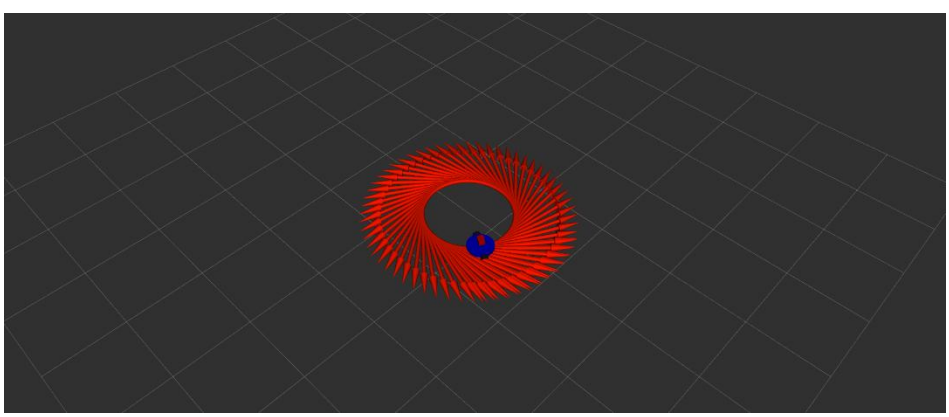
Phase 2: Precision Orbiting Logic

Once the robot is within the proximity threshold, it transitions to orbiting. The goal is to maintain a specific radius (R) so the entire height of the object (h) fits perfectly within the vertical Field of View (θ) of the camera:

$$R = \frac{h}{2\tan(\theta/2)}$$

To maintain a perfect circle, the robot calculates an ideal orbiting angular velocity ($\omega_{orbit} = v / R$). It then applies continuous PID corrections based on two error variables:

1. Distance Error: The difference between the current distance and target radius.
2. Heading Error: The deviation from keeping the object exactly at 90° (perpendicular to the robot's left side).



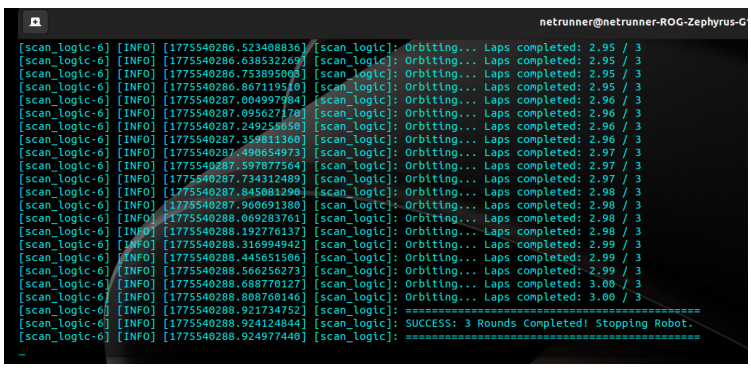
Phase 3: Lap Tracking and Stop Logic

To ensure the scanning process concludes automatically rather than running indefinitely, an angular integration method was implemented to track completed laps.

- Integration: During the orbiting phase, the script calculates the time elapsed between sensor callbacks (dt). It integrates the commanded angular velocity over time to track the total radians turned:

$$\theta_{total} = \Sigma\ (|\omega_z| \times dt)$$

- Termination Condition: One complete lap is exactly 2π radians. The robot is programmed to execute exactly 3 laps (6π radians).
- Halting: The moment $\theta_{total} >= 6\pi$, a software interrupt is triggered. The control node commands cmd.linear.x = 0.0 and cmd.angular.z = 0.0, permanently locking the robot in a safe, parked state, indicating the 3D scanning procedure is fully complete.



5. System Architecture & Final Product

The final product is a fully integrated, single-command launch system (qbot_bringup.launch.py) that initializes the following components simultaneously:

- Simulation Environment (Ignition Gazebo): A custom world (scan_world.world) featuring a central target cylinder (1.0m height, 0.2m radius) and optimal lighting for camera sensors.
- Robot Model (URDF/SDF): The Qbot is equipped with a differential drive plugin, a 2D LiDAR scanner (sensor_mount_link), and an RGBD camera.
- Communication Bridge: ros_gz_bridge translates Ignition topics (/scan, /cmd_vel, /tf, /joint_states) into native ROS 2 topics.
- Mapping Engine: SLAM Toolbox runs in online synchronous mode, subscribing to the /scan and /tf topics to dynamically generate a 2D occupancy grid map of the environment as the robot moves.
- Visualization: RViz2 is configured to visualize the TF tree (map → odom → base_link), live laser scan data, and the generated SLAM map.

6. Challenges Encountered & Solutions

Issue 1: Sensor Planar Limitations

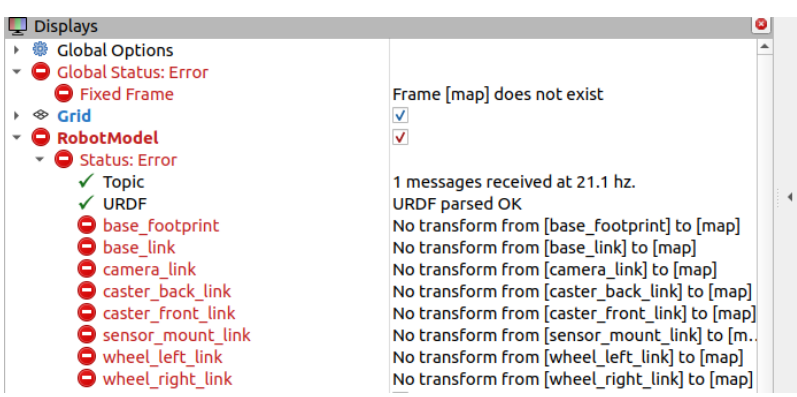
- The Problem: The 2D LiDAR fires a flat horizontal beam parallel to the ground (at Z = 0.14m). It was physically incapable of seeing the "top edge" of the 1.0m tall cylinder, meaning the robot could not dynamically calculate the object's height on the fly using just the laser.
- The Solution: We recognized the limitation of 2D scanning planes versus 3D objects. We mathematically decoupled the height calculation by hardcoding the object height (h = 1.0) into the Python script, allowing the LiDAR to focus solely on planar distance tracking while assuming the vertical dimensions.

Issue 2: Tracker Drifting and "Spiraling"

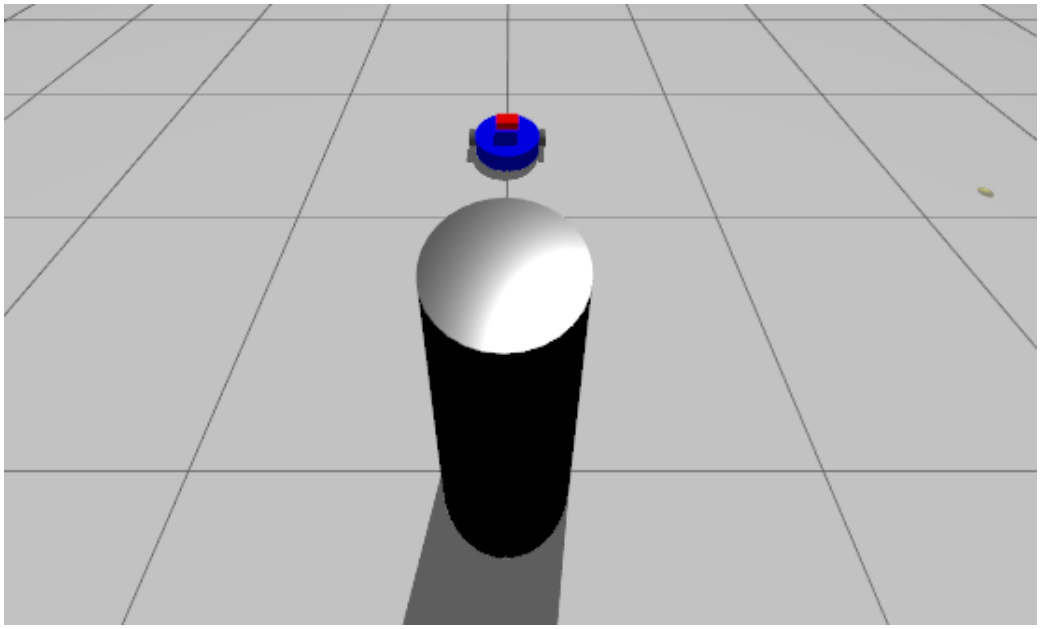
- The Problem: When the robot was initialized, it would occasionally lock onto the distant walls of the Gazebo environment (4.38m away) instead of the target object. This caused the robot to attempt to orbit the entire room, spiraling away from the center. Furthermore, the robot originally spawned facing away from the target.
- The Solution: We implemented two fixes:
    1. World File Correction: Modified the spawn orientation (Yaw = 180° or 3.14159 rad) so the robot spawned looking directly at the target.
    2. Software "Blinders": Added a filtering array to scan_logic.py (ranges < 2.0). This forced the robot to completely ignore any LiDAR returns further than 2.0 meters, guaranteeing it would only lock onto the central object and never the room's walls.

Issue 3: Broken TF Tree and RViz Errors

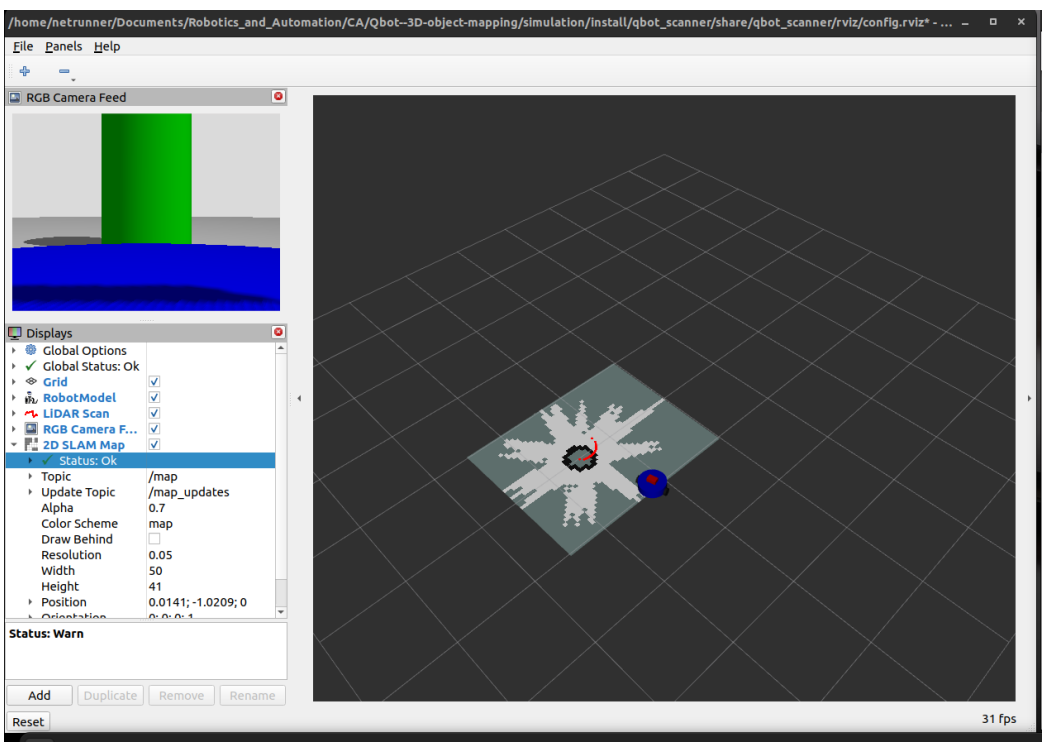
- The Problem: RViz displayed severe errors (Red links) and failed to show the robot model or map. The Fixed Frame was set to map, but without a mapping node running, the map → odom transform did not exist.
- The Solution: Integrated the SLAM Toolbox directly into the master qbot_bringup.launch.py file. This ensured that the moment the simulation started, the map frame was broadcasted, completing the TF tree and allowing RViz to visualize the environment perfectly without requiring secondary terminals.



7. Simulation Results



Initial spawn of Qbot facing the target cylinder in Ignition Gazebo.



RViz2 interface showing the completed TF tree, laser scan points bounding the cylinder, live camera feed and the SLAM map generating in the background.

- Video [Simulation] - Screen recording of the robot successfully approaching the target from 1.3m, locking onto the exact 0.85m calculated radius, and executing a smooth, continuous orbit.

8. Contribution

- **Dineth Sankalpa**: Handled the major workload for Week 2. Since the focus shifted heavily to simulation, He successfully built the new ROS 2 qbot_scanner package from scratch, developed the custom Ignition Gazebo environments, integrated the SLAM toolbox for live 2D mapping, and verified the complete simulation pipeline.
- **Group Effort & Learning Curve**: As the simulation phase is now completely finished, the entire team is currently navigating a steep learning curve regarding Raspberry Pi architecture and physical hardware programming. All three members (Dineth, Sanjuna, and Binara) spent the latter part of the week researching and preparing for the Raspberry Pi 4 setup. Next week, the entire team will collaborate on porting our finished logic to the physical bot. For the next week the heavy lifting will be upon the Binara and Sanjuna as they responsible for the hardware implementation and point cloud generation.

9. Goals for Next Week (Week 3)

Based on the new, accelerated timeline we established in your Week 2 report, the next 4 weeks (Weeks 3-6) are entirely dedicated to **Hardware Implementation & Testing**.
Since Week 3 is the kickoff for this phase, here are your specific objectives:

Week 3 Objectives: Initial Hardware Integration

1. Raspberry Pi 4 Configuration & Networking
Flash the Raspberry Pi 4 with Ubuntu 22.04 and install ROS 2 Humble.
Establish the distributed Wi-Fi ROS 2 network between the Raspberry Pi (on the Qbot) and your remote visualization PC.
Verify basic message passing (pinging and ROS Domain ID configuration) between the two machines.

2. Physical Sensor & Actuator Interfacing
Connect the physical Kobuki Qbot base to the Raspberry Pi and verify actuator control (e.g., sending simple cmd_vel commands to move the wheels).
Interface the newly acquired physical 2D LiDAR and the RGB-D camera with the Raspberry Pi, ensuring raw data is being published to the ROS network.

3. Real-World Data Visualization in RViz
Successfully stream the real-world 2D LiDAR data and TF (Transform) tree data from the physical robot to the remote PC.
Visualize the physical robot's movements and live laser scans in RViz on the remote PC.